

SZÁMÍTÓGÉP ARCHITEKTÚRÁK

Utasításrendszer szintje (ISA)

Tanács Attila

Architektúra szintek

5. Probléma orientált nyelv szintje
fordítás (fordító program)
4. **Assembly nyelv szintje**
fordítás (assembler)
3. Operációs rendszer szintje
részben értelmezés (operációs rendszer)
2. **Gépi utasítás szintje (ISA)**
*ha van mikroprogram, akkor értelmezés
(Kompatibilitás!)*
1. Mikroarchitektúra szintje
hardver
0. Digitális logika szintje

Fogalmak

• Gépi kód

- Numerikus gépi nyelv
- Az utasítások és az operandusok számok
- 1 utasítás 1 vagy több bájton kódolódik
- Elemi műveletek végrehajtására

• Assembly nyelv

- A numerikus gépi nyelv szimbolikus formája
- **Mnemonic**
 - emlékeztető kód a numerikus utasítások helyett
- Szimbolikus nevek és címek
- Makrók
- Feltételes fordítás
- ...

• Utasítások végrehajtása az 1950-es évek gépein

- <https://prog.hu/hirek/6576/video-programozas-1951-ben-tortenelem-retro-lyukkartya>

```
*****
* FUNCTION: INITA - Initialize ACIA
* INPUT: none
* OUTPUT: none
* CALLS: none
* DESTROYS: acc A
```

0013		RESETA	EQU	%00010011	
0011		CTLREG	EQU	%00010001	
C003	86 13	INITA	LDA A	#RESETA	RESET ACIA
C005	B7 80 04		STA A	ACIA	
C008	86 11		LDA A	#CTLREG	SET 8 BITS AND 2 STOP
C00A	B7 80 04		STA A	ACIA	
C00D	7E C0 F1		JMP	SIGNON	GO TO START OF MONITOR

Cím

Gépi
kód

Szimbólumok + mnemonikok

Megjegyzés

Kód forrása: [Thinking Soul](#)

**First programming language
was introduced in 1956**
People before 1956 :



ifunny.co

Meme forrása: [ifunny.co](#)

Mikroarchitektúra kapcsolat

- **Utasítás-végrehajtás hardveres megvalósítása**
 - 1 mikroutasítás hajtja végre (RISC-nél szokásos)
 - Jellemzően több mikroutasítás valósítja meg (CISC értelmezés, MIC-1 IJVM példa)
- **ISA szint (gépi kód)**
 - Hosszú időre (évtizedek) szóló tervezés a kompatibilitás biztosítására
 - Az alatta lévő szinten található mikroarchitektúra szint megvalósítása idővel változhat, de szoftveres szempontból ez nekünk mindegy!

Java fordítása IJVM-re

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

i = j + k;	1	ILOAD j	// i = j + k	0x15 0x02
if (i == 3)	2	ILOAD k		0x15 0x03
k = 0;	3	IADD		0x60
else	4	ISTORE i		0x36 0x01
j = j - 1;	5	ILOAD i	// if (i == 3)	0x15 0x01
	6	BIPUSH 3		0x10 0x03
	7	IF_ICMPEQ L1		0x9F 0x00 0x0D
	8	ILOAD j	// j = j - 1	0x15 0x02
	9	BIPUSH 1		0x10 0x01
	10	ISUB		0x64
	11	ISTORE j		0x36 0x02
	12	GOTO L2		0xA7 0x00 0x07
	13	L1: BIPUSH 0	// k = 0	0x10 0x00
	14	ISTORE k		0x36 0x03
	15	L2:		

Java programrészlet
(magasszintű)

Java Assembly nyelvű program
(Assembly szint)

Hexadecimális IJVM
(gépi kód)

Fogalmak

- **Assembler**

- A fordító, amely assembly nyelvről gépi kódra fordít

- **Disassembler**

- Gépi kódból mnemonik kód listázása
- Vigyázni kell, hogy a listázás kezdőcíme valóban utasításhatáron legyen!
- Ha szimbólumlista elérhető, akkor Assembly-szerű lista kapható

Assembly nyelv

- **Jellemzők**

- Minden utasításnak egyetlen gépi utasítás felel meg
- Architektúránként különböző Assembly nyelv!
 - Pl. Intel, UltraSPARC, RISC-alapú architektúrák
 - Nincs a magasszintű nyelveknél tapasztalható portabilitás!

- **Hátrányok**

- Nehézkes, időigényes
- Sok hibalehetőség
- Hosszadalmasabb hibakeresés, karbantartás

- **Előnyök**

- Hatékonyság
- A hardver teljes elérhetősége
 - Bizonyos regiszterek magasszintű nyelvekből nem használhatók
 - Hardver közvetlen elérése

Fordítás, szerkesztés

- **Fordító (Compiler)**

- *Forrásnyelvből célnyelvre alakít, pl.:*
 - C++, C -> tárgykód (.o/.obj)
 - **Assembly -> tárgykód (.o/.obj)**
 - Java -> class fájl
 - C -> Assembly (.asm)
 - FORTRAN -> C

- **Szerkesztő (Linker)**

- *Tárgykódok összeszerkesztése futtatható állománnyá*
 - Külső hivatkozások feloldása
 - Virtuális címek feloldása
 - Címterek összefésülése (relokáció)
 - ...

Assembly program fordítása

- **Jellemzően kétmenetes fordítók (*compiler*)**
 - **Forráskódból tárgykód készítése**
 - Tárgykód: lefordított utasítások + szerkesztőnek szóló információk
 - **Első menet**
 - Szimbólumtábla felépítése
 - Címkék, változók, literálok, makrók, ... feljegyzése
 - Utasítás-helyszámláló segítségével
 - **Második menet**
 - Tárgykód elkészítése
 - Kimeneti fordítási listák készítése
 - Szerkesztőnek szóló utasítások tárgykódba helyezése

Assembly program fordítása

- **Szerkesztés (*linker*)**

- Tárgykódok futtatható kóddá szerkesztése
 - Az azonos nevű és osztályú szegmens szeletek egymáshoz illesztése a szegmens szeletek definíciójában megadott módon,
 - egy csoportba sorolt szegmensek egymás után helyezése,
 - relokáció elvégzése,
 - külső hivatkozások feloldása.
- Típusok
 - Statikus
 - A tárgykód a futtatható állományba kerül
 - Dinamikus
 - Nagyméretű programokban bizonyos eljárások csak nagyon ritkán szükségesek. Ezeket nem kell statikusan a programhoz szerkeszteni. A szerkesztés futási időben történik meg (operációs rendszer).
 - .dll (*dynamic link library*), .so (*shared object* – UNIX) fájlok

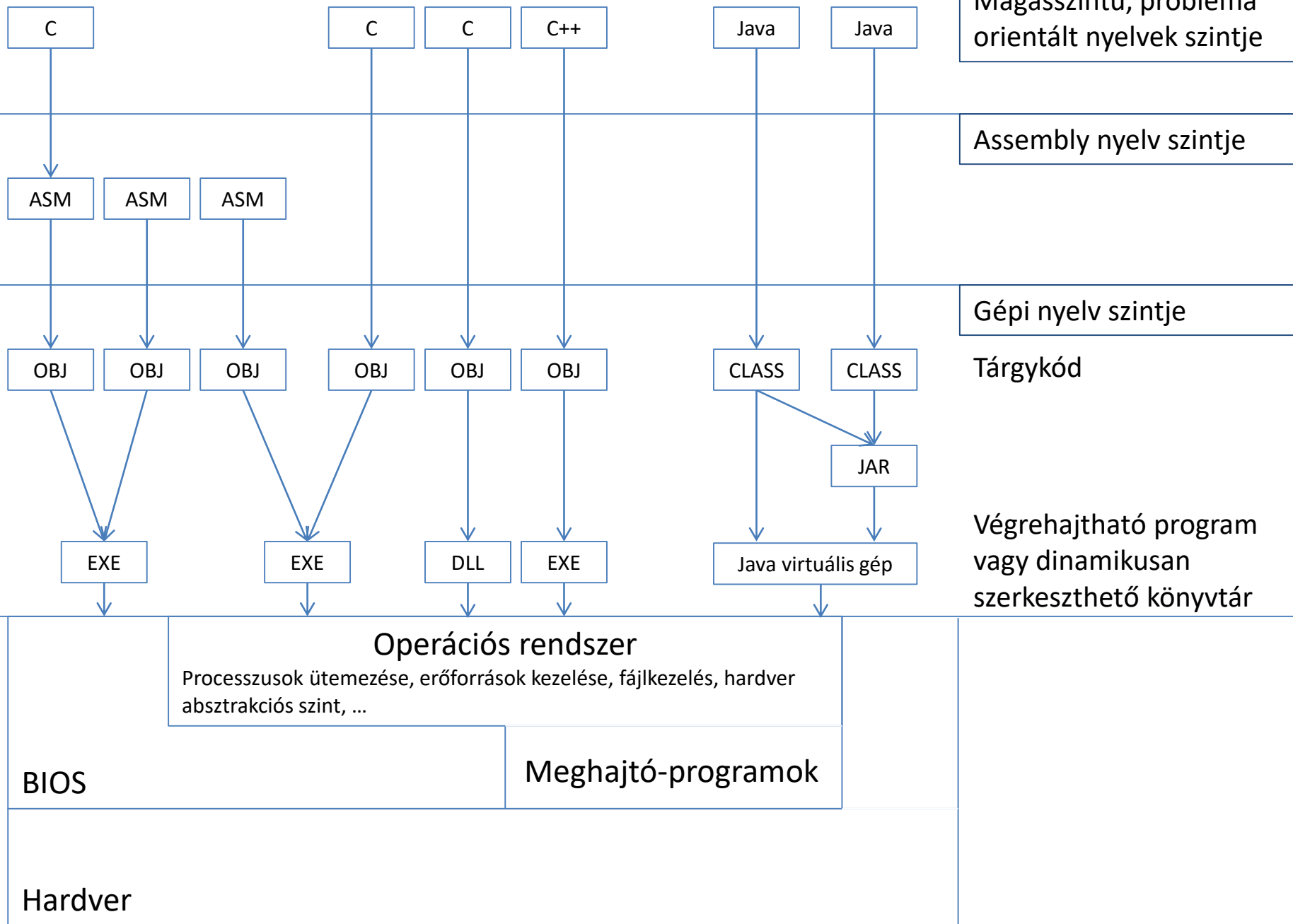
Fájlformátumok, kiterjesztések

- **Tárgykód**

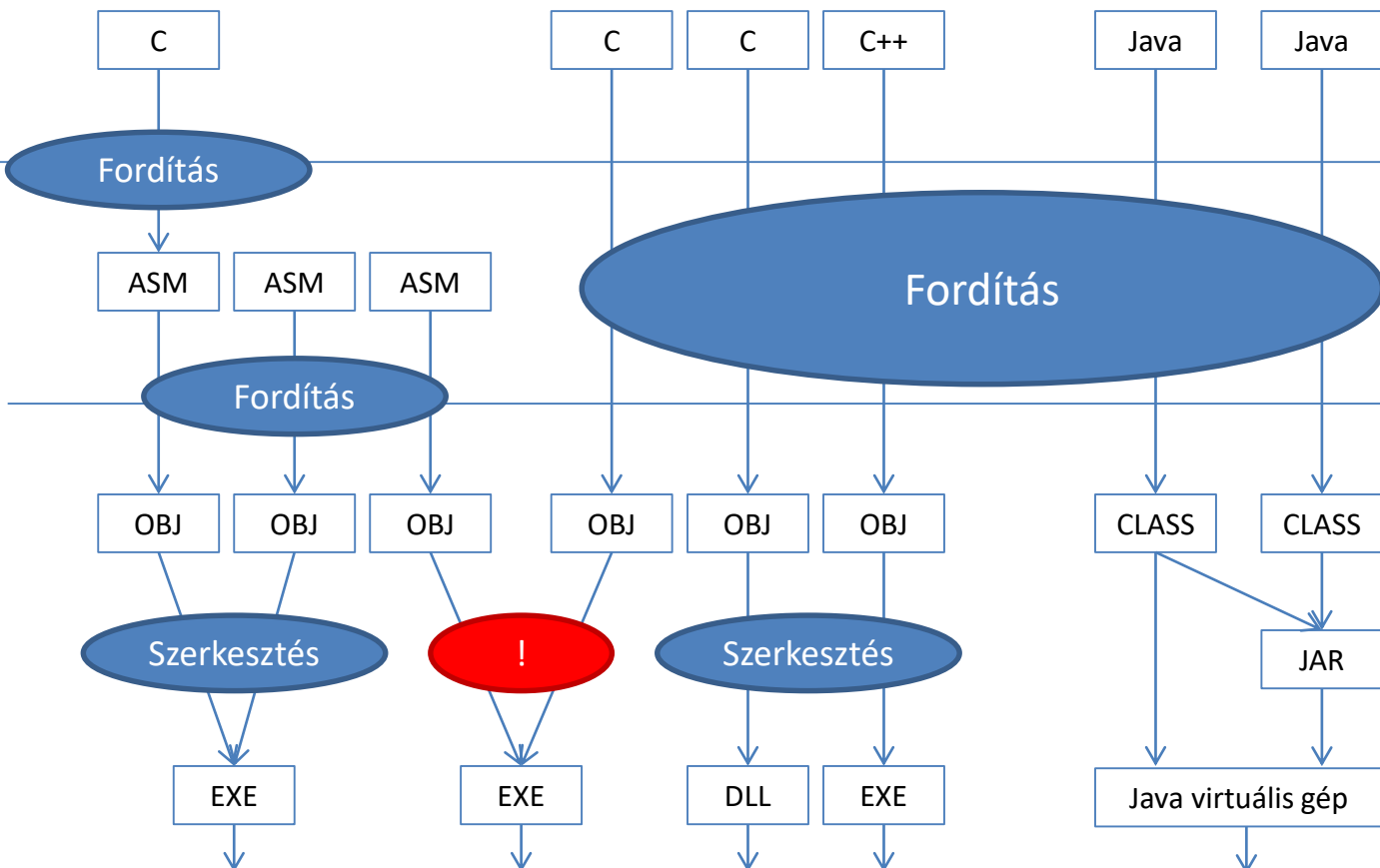
- .obj: DOS, Windows
- .o: Unix, Linux, Cygwin, ...
- Fordító-specifikus bináris formátum!
 - Azonos kiterjesztés még nem jelenti azt, hogy összeszerkeszthetők!

- **Végrehajtható (futtatható)**

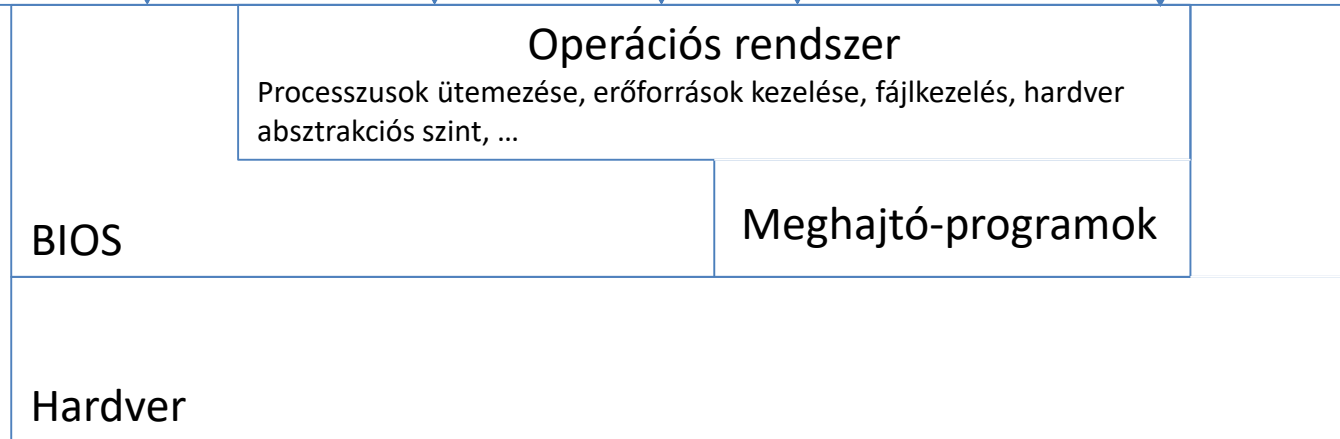
- .com: kis méretű DOS alkalmazások
- .exe: DOS, Windows formátum (többféle szerkezet!)
- .dll: dinamikusan szerkeszthető (Windows)
- .so: dinamikusan szerkeszthető (Unix, Linux, ...)
- .a: statikusan szerkeszthető tárgykód gyűjtemény (Unix, Linux...)
- Unix-alapú rendszereknél nincs futtatható kiterjesztés, azt a fájl jogosultsága mondja meg



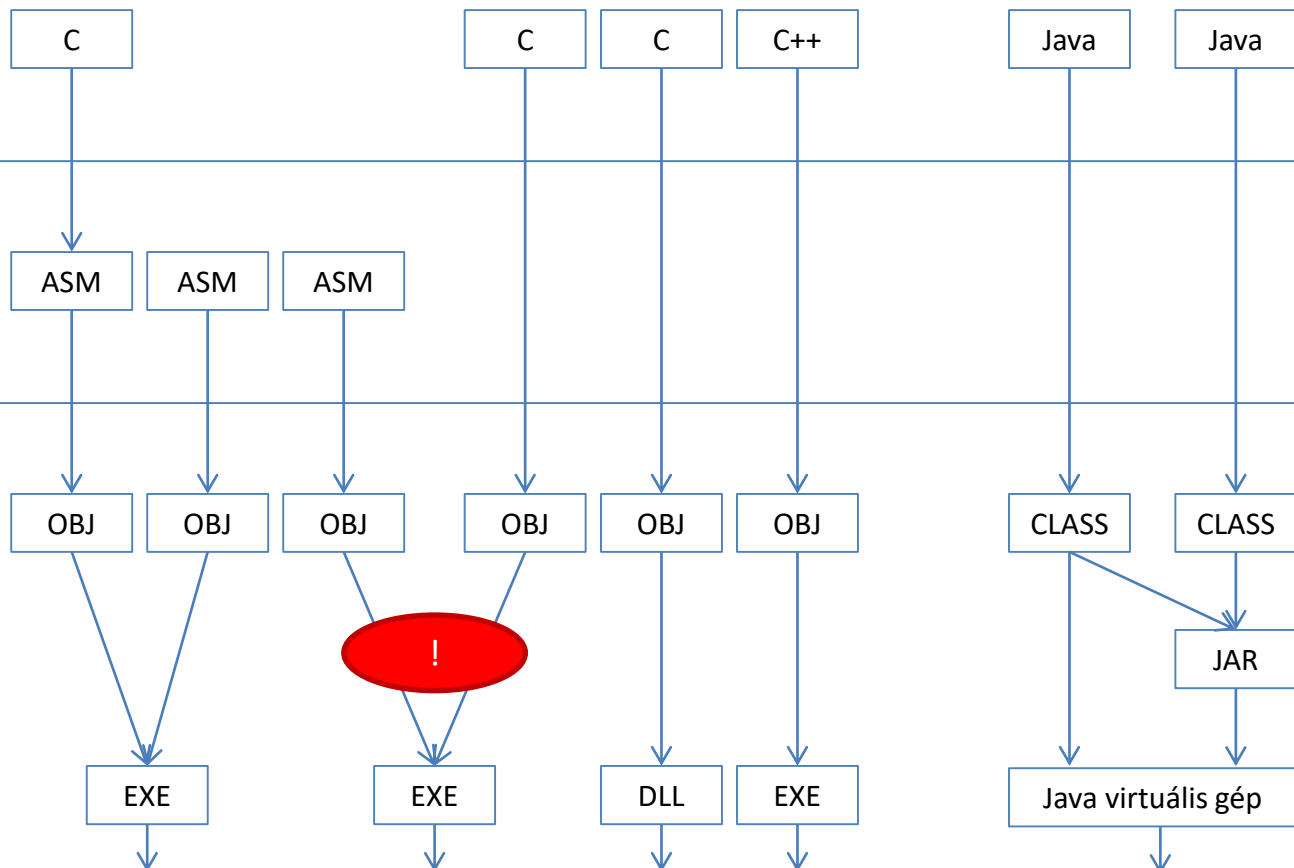
Az ábrát nem kell megtanulni!



Különböző forrásnyelvekből származó tárgykódok akkor szerkeszthetők össze, amennyiben megegyező formátumúak, valamint egyeznek a hívási konvenciók, paraméterátadások, a memóriamodell, ...



Az ábrát nem kell megtanulni!



Assembly:
hardverközeli és/vagy
hatékony implementációt
igénylő részek

Magas szintű nyelv:
GUI, vezérlés, Assembly
betétek hívása

Operációs rendszer

Processzusok ütemezése, erőforrások kezelése, fájlkezelés, hardver
absztrakciós szint, ...

BIOS

Meghajtó-programok

Hardver

Az ábrát nem kell megtanulni!

C → Assembly fordítás példa

- C forráskód

```
#include <stdio.h>
int main( int argc, char *argv[] )
{
    int a, b, sum;
    a = 5;
    b = 7;
    sum = a + b;
    printf( "Osszeg: %d\n", sum );

    return 0;
}
```

A példát nem kell megtanulni!

Generált Assembly kód (Intel Ubuntu Linux)

```
.file "sum.c"
.section .rodata
.LC0:
.string "Osszeg: %d\n"
.text
.globl main
.type main, @function
main:
    pushl    %ebp
    movl     %esp, %ebp
    andl     $-16, %esp
    subl     $32, %esp
    movl     $5, 28(%esp)
    movl     $7, 24(%esp)
    movl     24(%esp), %eax
    movl     28(%esp), %edx
    leal     (%edx,%eax), %eax
    movl     %eax, 20(%esp)
    movl     $.LC0, %eax
    movl     20(%esp), %edx
    movl     %edx, 4(%esp)
    movl     %eax, (%esp)
    call     printf
    movl     $0, %eax
    leave
    ret
.size      main, .-main
.ident     "GCC: (Ubuntu/Linaro 4.5.2-8ubuntu4) 4.5.2"
.section   .note.GNU-stack,"",@progbits
```

Generált Assembly kód (SUN Sparc)

```
.file "sum.c"
gcc2_compiled.:
.section ".rodata"
    .align 8
.LLC0:
    .asciz "Osszeg: %d\n"
.section ".text"
    .align 4
    .global main
    .type    main,#function
    .proc    04
main:
    !#PROLOGUE# 0
    save     %sp, -128, %sp
    !#PROLOGUE# 1
    st       %i0, [%fp+68]
    st       %i1, [%fp+72]
    mov      5, %o0
    st       %o0, [%fp-20]
    mov      7, %o0
    st       %o0, [%fp-24]
    ld       [%fp-20], %o0
    ld       [%fp-24], %o1
    add      %o0, %o1, %o0
    st       %o0, [%fp-28]
    sethi    %hi(.LLC0), %o1
    or       %o1, %lo(.LLC0), %o0
    ld       [%fp-28], %o1
    call     printf, 0
    nop
    mov      0, %i0
    b        .LL2
    nop
.LL2:
    ret
    restore
.LLfe1:
    .size     main, .LLfe1-main
    .ident    "GCC: (GNU) 2.95.3 20010315 (release)"
```

A példát nem kell megtanulni!

Tárgykód (Intel Borland C fordító)

The screenshot shows a debugger window titled 'Lister - [D:\Study\A...]' with a menu bar (File, Edit, Options) and a status bar (100 %). The main window is divided into two panes. The left pane displays assembly code with addresses, hex values, and mnemonics. The right pane displays the contents of a file named 'sum.c'.

Assembly Code (Left Pane):

```

00000000 80 07 push ebp
00000010 42 61 mov ebp, esp
00000020 31 01 mov eax, 5
00000030 25 41 mov edx, 7
00000040 42 61 ?live1@32: ; EAX = a
00000050 63 61 add edx, eax
00000060 28 01 mov eax, edx
00000070 61 61 ?live1@48: ; EAX = a, EDX = b
00000080 65 51 add edx, eax
00000090 28 D1 mov eax, edx
000000A0 63 61 ?live1@64: ; EAX = sum
000000B0 66 71 push eax
000000C0 3A 51 push offset s@
000000D0 69 61 call _printf
000000E0 68 01 add esp, 8
000000F0 6F 71 xor eax, eax
00000100 6C 71 pop ebp
00000110 00 E1 ret

```

File Contents (Right Pane):

```

sum.c
Borland C++ 5.5.
1 . 1 1 sum.c.5
%a .) é (Ú(c:\
Borland\Bcc55\in
clude\ nfile.h .
( é (Ú( c:\Borl
and\Bcc55\includ
e\ null.h .( é
(Ú( c:\Borland\B
cc55\include\ de
fs.h .* é (Ú(c\
:\Borland\Bcc55\
include\ stddef.
h .( é (Ú( c:\B
orland\Bcc55\inc
lude\stdio.h ..
é.%a|sum.c .L
é -( |TEXT|COD
E |DATA|DATA-DG
ROUP|BSS|BSS .
@# . 1 . 1 1 1
. 1 1 1 1 1
' 1 1 1 1 1
gv_ . _printf .
1 1 1 1 1
. U<ě, | 1
1 1 1 1 1
.Ä3R]Ä t. ä!!T;#
1U 1 1 1 1 1
: %d. < 1

```

A yellow arrow points from the assembly code to the file's contents.

Fájl tartalma:
kód, adat és
járulékos (pl.
szerkesztőnek
szóló)
információk.

Középen hexa
számokként,
jobb
oszlopban
karakterkódok-
ként.

A példát nem kell megtanulni!

ISA szint

- **Összekötő kapocs a szoftver és a hardver között**
 - Több magasszintű nyelven legyen programozható a számítógép
 - Közvetlenül az ISA-szint hajtható végre
 - Szükséges egy ISA-szintre történő fordítás
- **ISA-szint tervezése**
 - Hardver oldalról: mi valósítható meg (költség)hatékonyan
 - Szoftver oldalról: milyen utasításokra lenne szükség a magasszintű nyelvekről fordításkor
 - Hosszú időre szóló terv kell!
- **Kompatibilitás**
 - Kulcsfontosságú!
 - Nem lehet pár havonta/évente a szoftverrendszereket újratervezni
 - Visszafelé kompatibilitás
 - Az új futtatni tudja a régit, de fordítva nem (kell) teljesül(jön)
 - Új mikroarchitektúra szint is bevezethető így

ISA szint

- **Fő komponensei**

- Regiszterkészlet
- Memóriamodell
- Felhasználható utasítások és adattípusok

- **Nem része**

- Mikroarchitektúra tényleges megvalósítása
 - Csővezetékes, szuperskaláris, ...
- De ezek az ismeretek felhasználhatók optimális gépi kódú programok készítéséhez/fordításához
 - Pl. egész és lebegőpontos ALU-val rendelkező szuperskaláris gép egyszerre végre tud hajtani két ilyen műveletet: érdemes az utasítások sorrendjét így megadni a kódban

ISA szint

- **Védelem nélküli rendszerek**

- Minden utasítás végrehajtható a felhasználói programokban, és
- „szabadon garázdálkodhat” (akár az operációs rendszer kódját is felülírhatja)
- Pl. DOS

- **Védett üzemmódok**

- CPU-szintű hardveres védelem (utasítás-végrehajtás, kódterület)
- **Felhasználói mód**
 - Bizonyos utasítások végrehajtásának tiltása
 - Pl. gyorsítótár manipulálása
- **Kernelmód**
 - Operációs rendszer futtatásához használt
 - Nincs korlátozás

Memóriamodellek

- **Korábbi előadásanyagban tárgyalásra került**
 - Cellák, cellaméret
 - Bájtok, szavak
 - Bájtok értékeinek értelmezése
 - Adat, kód
 - Igazítás

Memóriamodellek

- **Címtartomány**

- **Lineáris cím**

- Bájtok sorszámozása 0-tól
 - Bájt címe az indexe

- **Szegmentált cím**

- BÁZISCÍM : ELTOLÁS (Szegmens : Offszet) alakban
 - Szegmensen belüli relatív címek használhatók
 - Könnyebb relokáció
 - A szegmens áthelyezhető a memória más részére a címzések és vezérlésátadások módosítása nélkül
 - 16 bites architektúrák esetén használatos (pl. Intel 8086)

ISA regiszterek

- **Nagyon gyors elérésű memóriarekeszek**
 - Végrehajtás vezérlésére
 - Aritmetikai, logikai műveletek elvégzésére
 - Eredmények ideiglenes tárolására
- **Mikroarchitektúra kapcsolat**
 - Nem minden mikroarchitektúra regiszter jelenik meg az ISA szinten!
 - Pl. rendszerint nincs TOS, MAR, ...
 - De általában látható a PC, SP, ...
 - Az ISA szint regiszterkészlete elérhető a mikroarchitektúra szinten
 - Mivel ott kerülnek megvalósításra

ISA regiszterek

- **Speciális célú**

- Utasításszámláló, veremmutató, flag regiszter, ...
- Csak rögzített feladatra használhatók
- Csak speciális utasításokkal módosíthatók

- **Általános célú**

- Gyakran használt adatok tárolására, gyors elérésére
- Felcserélhető regiszterek
 - Teljesen ekvivalensek
- Alkalmazási konvenciók
 - Könyvtári alkalmazásokban rögzített szerep miatt (paraméterátadás)
 - Célszerű betartani és alkalmazni
- Speciális szereppel rendelkezők
 - Pl. szorzás eredménye mindig 1 megnevezett regiszterbe kerül
 - Nem felülbíráható viselkedés

ISA regiszterek

- **Jelzők vezérlőregisztere**

- Flag vagy PSW (*Program Status Word*) regiszter
- Bitek, mint feltételkódok
 - Minden ALU művelet beállítja őket
 - Más utasítások is állíthatják, értéküket vizsgálhatják
 - Elsősorban **feltételes elágazásoknál** használatosak
- Tipikus feltételkódok
 - N (negative): beállítva, ha az eredmény negatív volt
 - Z (zero): beállítva, ha az eredmény nulla volt
 - V (oVerflow): beállítva, ha volt túlcsordulás
 - C (Carry): beállítva, ha volt átvitel
 - ...

ISA utasítástípusok

- **Adatmozgató**
 - Memória és/vagy regiszterek között
 - Nem minden kombináció érhető el
 - Pl. általában legalább 1 operandus regiszter kell legyen
- **Aritmetikai és logikai**
 - Összeadás, kivonás, szorzás, osztás
 - Logikai műveletek, bitléptetés, bitforgatás
- **Összehasonlító**
 - FLAG biteket állítanak
- **Vezérlésátadó**
 - Feltételes és feltétel nélküli
 - Ugró és eljárashívó
 - Megszakításkezelő

Rendszerek áttekintése

- **Pentium 4 és Core i7 (Intel 80x86 család)**
 - Erőteljes visszafelé kompatibilitás
 - Összetett utasításrendszer
 - Elavult részek
- **UltraSPARC III és TI OMAP4430**
 - Betöltő/tároló, RISC architektúra
 - Fix méretű utasítások
- **Intel 8051 és ATmega168 mikrovezérlők**
 - Egyetlen üzemmód, egyszerű memóriamodell, egyszerű utasítások
- **RISC-V**
 - Nyílt forrású, ingyenesen használható ISA

Intel 80x86 processzorcsalád

A táblázatot nem kell megtanulni!

Lapka	Dátum	MHz	Tranz.	Mem.	Megjegyzés
I-4004	1971/4	0.108	2300	640	Első egylapkás mikroprocesszor
I-8008	1972/4	0.108	3500	16 KB	Első 8 bites mikroprocesszor
I-8080	1974/4	2	6000	64 KB	Első általános célú mikroprocesszor
I-8086	1978/6	5-10	29000	1 MB	Első 16 bites mikroprocesszor
I-8088	1979/6	5-8	29000	1 MB	Az IBM PC processzora
I-80286	1982/6	8-12	134000	16 MB	Védett üzemmód
I-80386	1985/10	16-33	275000	4 GB	Első 32 bites mikroprocesszor, virtuális 8086 üzemmód
I-80486	1989/4	25-100	1.2M	4 GB	8 KB beépített gyorsítótár
Pentium	1993/5	60-233	3.1M	4 GB	Két csővezeték, MMX
P. Pro	1995/3	150-200	5.5M	4 GB	Két szintű beépített gyorsítótár
P. II	1997/5	233-400	7.5M	4 GB	Pentium Pro + MMX
P. III	1999/2	650-1400	9.5M	4 GB	SSE utasítások 3D grafikához
P. 4	2000/11	1300-3800	42M	4 GB	Hyperthreading + több SSE
Core Duo	2006/1	1600-3200	152M	2 GB	Két mag egy lapkán
Core	2006/7	1200-3200	410M	64 GB	64 bites, négymagos architektúra
Core i7	2011/1	1100-3300	1160M	24 GB	Integrált grafikus processzor

P4 és Core i7 32 bites üzemmódjai

- **Valós mód**

- 8086/8088-ként működik
- Max. 1 MB memória címezhető

- **Védett mód**

- A CPU lehetőségeinek teljes kihasználása
- Négy privilegizált szint (PSW bitek vezérlik)
 - 0-s szint: kernel mód (operációs rendszer)
 - 1-es, 2-es szint: ritkán használt
 - 3-as szint: felhasználói programok

- **Virtuális 8086 mód**

- Párhuzamosan több valós módú virtuális gép futthat
- Védett módú környezet vezérli
- Ha „lefagy” a virtuális gép, visszatérhetünk az indító környezethez

Üzem mód		Szükséges OS	Alkalmazás újrafordítás	Alapbeállítások (bit)		Regiszter kiterjesztés	Ált. célú reg. méret
				Cím	Operandus méret		
Long	64-bit	64-bites	Igen	64	32	Igen	64
	Kompatibilitási		Nem	32	32	Nem	32
				16	16		16
Legacy	Védett mód	32-bites	Nem	32	32	Nem	32
				16	16		
	Virtuális 8086			16	16		16
	Valós mód	16-bites					

P4 és Core i7 32 bites regiszterei

Általános célú regiszterek

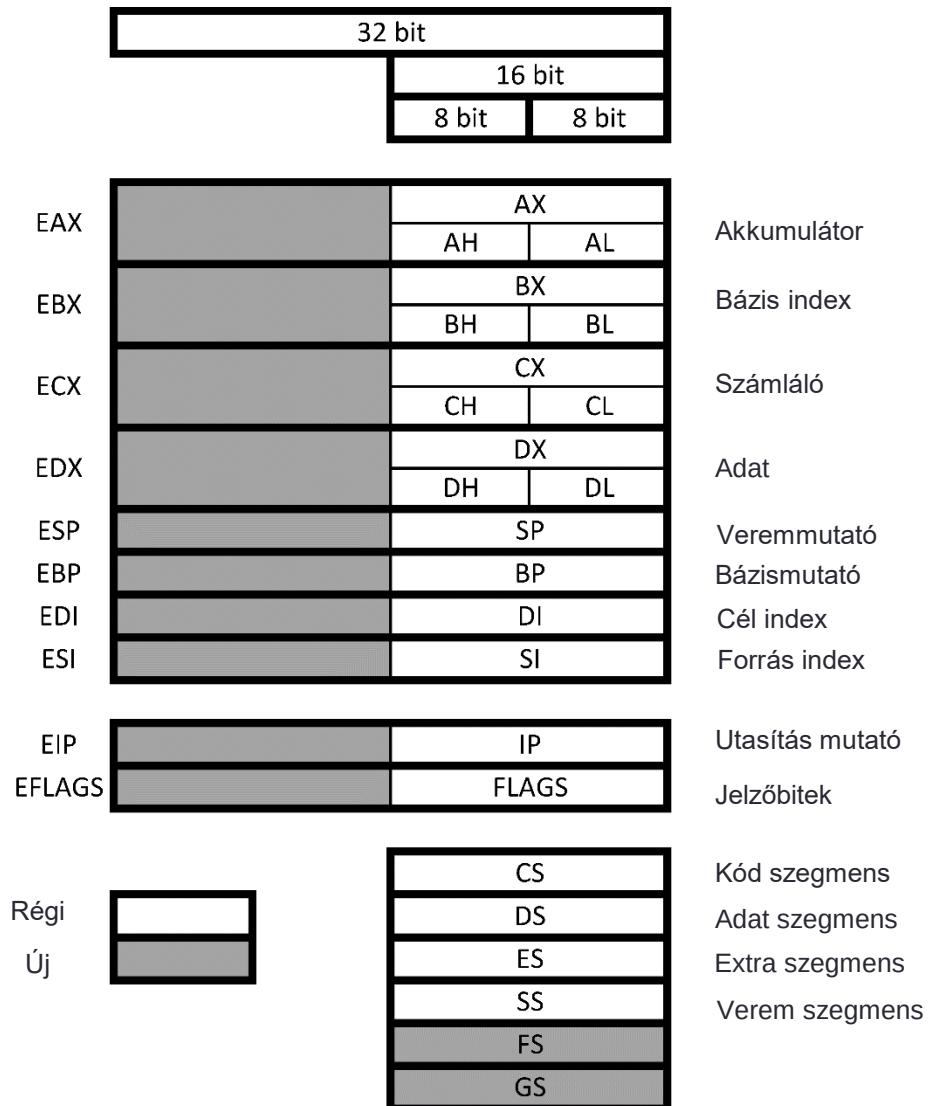
- 32 bitesek
- EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
- Alsó 16 bit, és annak alsó/felső bájtja külön elérhető
 - EAX; AX; AH; AL

Speciális célú regiszterek

- 32 bitesek
- EIP, EFLAGS (PSW)

Szegmens regiszterek

- 16 bitesek
- CS, SS, DS, ES, FS, GS



P4 és Core i7 32 bites regiszterei

- **Általános célú regiszterek**

- EAX: Akkumulátor
Egyedi szerep szorzásnál és osztásnál
- EBX: Bázis index
- ECX: Számláló
Egyedi szerep bitléptető, sztring és ismétléses műveleteknél
- EDX: Adat
Egyedi szerep szorzásnál és osztásnál
- EBP: Bázis mutató
Egyedi szerep verem indexelt címezésére
- ESI: Forrás index
Egyedi szerep sztring műveleteknél
- EDI: Cél index
Egyedi szerep sztring műveleteknél
- ESP: Veremmutató
Verem műveletek állítják. Erős speciális jellege miatt általános célra nem célszerű használni.

P4 és Core i7 32 bites regiszterei

- **Speciális célú regiszterek**

- EIP: Utasítás számláló (Csak vezérlésátadással írható felül)
- EFLAGS: CPU állapotát jelzőbitek összessége
 - C (Carry): Átvitel előjel nélküli műveleteknél
 - P (Parity): Az eredmény alsó 8 bitjének paritása
 - A (Aux Carry): Átvitel a 3. és 4. bitek között (BCD számoknál)
 - Z (Zero): 1 (igaz), ha az eredmény 0, különben 0 (hamis)
 - S (Sign): Az eredmény legmagasabb helyiértékű bitje (előjel)
 - T (Trap): 1: debug mód, 0: automatikus
 - I (Interrupt): 1: maszkolható megszakítás engedélyezve, 0: tiltva
 - D (Direction): Sztring műveletek iránya 1: csökkenő, 0: növekvő
 - O (Overflow): Előjeles túlcsondulás

A jelzőbiteket nem kell megtanulni!

P4 és Core i7 32 bites regiszterei

- **Szegmens regiszterek**

- CS (Code Segment): kód szegmens
 - Csak ugró utasítással módosítható
- DS (Data Segment): adat szegmens
- ES (Extra Segment): (másodlagos) adat szegmens
- SS (Stack Segment): verem szegmens
- FS, GS: segéd szegmens regiszterek

- **Használatuk**

- **Védett üzemmódban** (ez a jellemző)
 - Szelektor index a szegmens leíró táblára
 - Tábla tartalma: szegmens fizikai kezdőcíme, mérete, hozzáférési jogosultságok
 - Szegmensen belül 32 bites címtartomány
 - Több operációs rendszer csak 1 lineáris címtartományt engedélyez
 - UNIX, Windows, ...

P4 és Core i7 32 bites regiszterei

• Szegmensregiszterek használata

Nem kell megtanulni!

• 16 bites 8086 üzemmódban (elavult!)

- 20 bites címbusz, de csak 16 bites regiszterek
- 1 regiszter nem elegendő a címzéshez, használjunk kettőt
 - SZEGMENS : OFFSZET alakban írjuk
 - Fizikai cím előállítása: a SZEGMENS értéke 16-tal (10H) szorzódik / 4 bittel balra tolódik / hexa 0 íródik utána és hozzáadódik az OFFSZET értéke
- Vagyis
 - Egy szegmens mérete maximálisan 64 KB lehet
 - Egy fizikai címnek több szegmentált címe létezik!
- Túlcsordulás kezelés nincs
 - FFFF:0010 az első (0.) memóriapozíciót jelenti!
- Az egyes szegmensek részben vagy teljesen átfedőek lehetnek!

SZEGMENS: 1234H
OFFSZET: 0012H

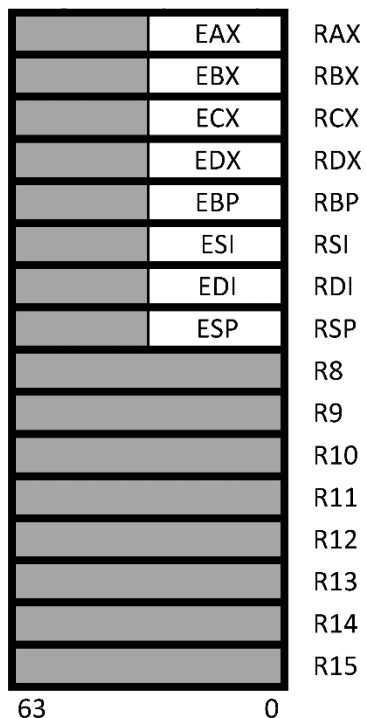
12340H	* 10H
+ 0012H	+ 0F.

12352H	

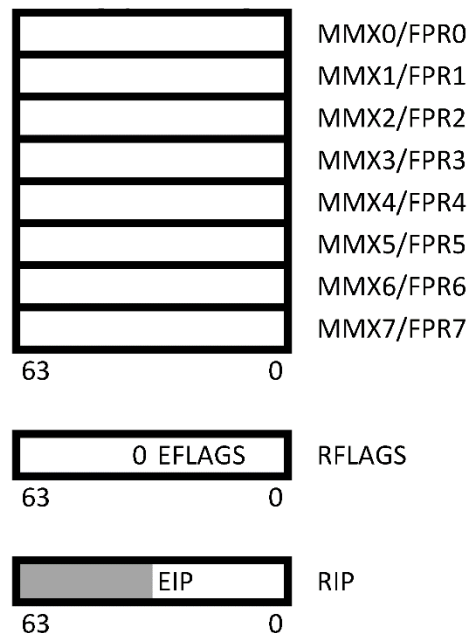
• Kis endián tárolás

Core i7 64 és 128 bites regiszterei

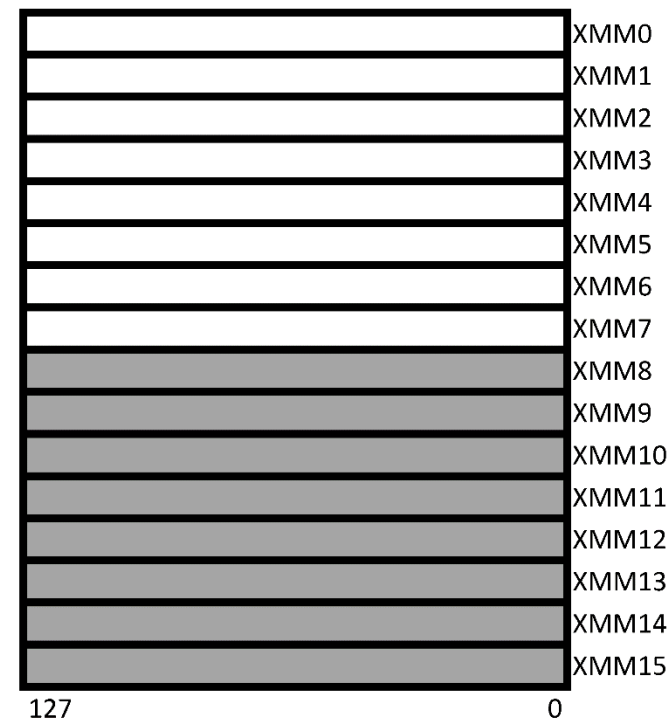
Általános célú
regiszterek (GPR-ek)



64 bites média és
lebegőpontos reg.



128 bites média regiszterek



Örökölt regiszterek, minden üzemmódban elérhetők
64-bites módban támogatott, kiterjesztett regiszterek

E = „Extended”
R = „Really extended”

UltraSPARC III ISA-szintje

- **Jellemzői**

- Az egyik első RISC architektúra leszármazottja (SPARC 1987)
- 64 bites működés

- **Memóriaszerkezet**

- 2^{64} bájt méretű lineáris tömb, jelenleg 44 bites címezhető tartomány
- Nagy endián bájtsorrend
 - PSW megfelelő bitjével kis endiánra is állítható

- **Regiszterek**

- 32 darab 64 bites általános célú regiszter (R0 – R31), konvenciók:
 - R0 (G0): hardveres 0 érték, minden tárolás eredménytelen
 - R1-R7 (G1-G7): Globális változók
 - R8-R13 (O0-O5): Hívott eljárás paraméterei
 - R14 (SP): Veremmutató
 - R15 (O7): Ideiglenes regiszter
 - R16-R23 (L0-L7): Aktuális eljárás lokális változói
 - R24-R29 (I0-I5): Bejövő paraméterek
 - R30 (FP): Aktuális veremkeret-mutató
 - R31 (I7): Aktuális eljárás visszatérési címe
- 32 darab lebegőpontos regiszter

UltraSPARC III

- **Regiszterablak**

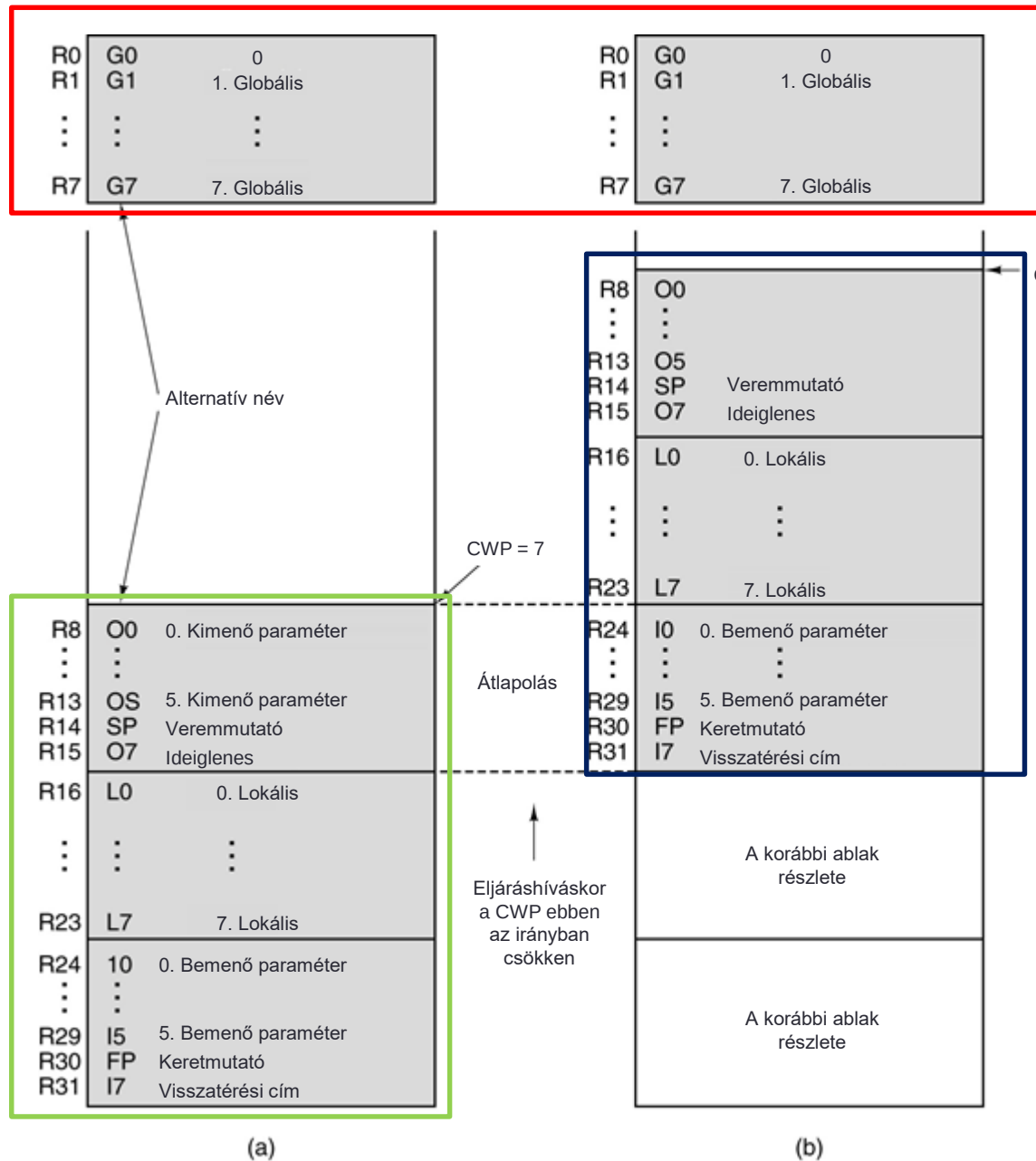
- Ténylegesen 32-nél több regiszter van
- De csak 32 látszik egyszerre
- CWP regiszter: aktuálisan mely regiszterek használhatók
 - Eljáráshíváskor veremszerű működés: új készlet nyílik
 - A globális regiszterek az új készletben is megtalálhatók lesznek

- **Probléma**

- Eljáráshívások egymásba ágyazásával elfogynak a regiszterek
- Ekkor a betelt készletet ki kell írni a memóriába
- Eljárásból visszatéréskor vissza kell olvasni
- Csak nem túl mély egymásba ágyazottság esetén hatékony

Minden
ablakban
látható

Hívás előtti
ablak



Hívás utáni
ablak
A kimenő
regiszterek,
bemenőkké
válnak

Csak az ábra lényegét kell tudni!

UltraSPARC III

- **Betöltő/tároló architektúra**

- Memória írás/olvasás csak regiszterből/regiszterbe történhet
- A többi utasítás csak regiszter vagy beépített adat operandussal rendelkezhet
- Az eredmény mindig regiszterbe kerül

TI OMAP4430 (ARMv7) ISA-szintje

- **Jellemzői**

- Mobil eszközökben (okostelefonok)
- ARM Cortex A9 processzor
- ARMv7 ISA
- 32 bites, RISC
- Tiszta ISA
 - Betöltő-tároló architektúra

- **Memóriaszerkezet**

- 2^{32} bájt méretű lineáris tömb a címtartomány
 - Kezd kevés lenni még mobil eszközökben is...
 - (ARMv8: már 64 bites architektúra)
- Kétféle endián üzemmód lehetőség
 - Rendszerindításkor a rendszer memória blokk tartalma határozza meg
 - Ez az információ mindenképpen kis-endián módban tárolódik

TI OMAP4430 (ARMv7)

• Regiszterek

- Program Status Register (PSR): jelzőbitek
- Általános célú: 16 darab 32 bites regiszter

Regiszter	Alternatív név	Funkció
R0 – R3	A1 – A4	Eljáráshívás paraméterei
R4 – R11	V1 – V8	Aktuális eljárás lokális változói
R12	IP	Eljárások közötti hívás regisztere: Ha $\pm 2^{25}$ bájt távolságnál messzebbre kell ugrani, ez tartalmazza a címet (32 bites utasításhosszba nem fér bele 32 bites cím az opkód mellé...)
R13	SP	Veremmutató
R14	LR	Link regiszter (aktuális függvény visszatérési címe)
R15	PC	Programszámláló (aritmetikai művelet operandusa is lehet!)

- Lebegőpontos: 32 darab 32 bites regiszter
 - 16 darab 64 bitesként is elérhetők! (Egyszeres és dupla pontosság)

Intel 8051 ISA-szintje

- **Beágyazott rendszerekben**
 - Egyetlen üzemmód, nincs védelmi hardver
 - Egyszerre 1 program fut
- **Memóriamodell**
 - 64 KB címtartomány programnak (ROM)
 - 64 KB címtartomány adatoknak (RAM)
- **Lehetséges megvalósítások**
 - Többféle lehet
 - 4 KB ROM, 128 bájt (!) RAM, lapkára integrálva (gyors elérés!)
 - 64 KB külső ROM, 64 KB külső RAM (lassabb, de nagyobb)
 - 64 KB külső RAM a programnak és az adatoknak
 - ...

Intel 8051

- **Általános regiszterkészlet**

- 4 elkülönülő regiszterkészlet
- Készletenként 8 darab 8 bites regiszter (R0 – R7)
 - PSW 2 bites mezője választja ki az aktuális készletet
- Gyors megszakításkezeléshez hasznos!
 - Elegendő a készletváltás, nem kell menteni és visszatölteni az értékeket

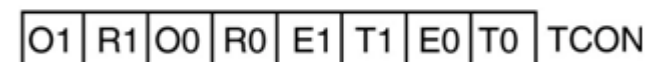
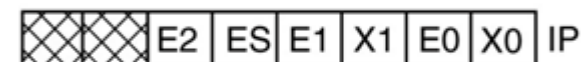
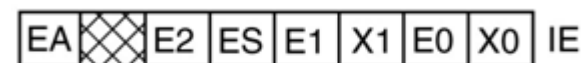
- **Memóriába ágyazottak**

- Memória első $8 \times 4 = 32$ bájtja
 - Memória 0. bájtja: 0. készlet R0 regisztere
 - Memória írása/olvasása a regiszter írással/olvasással egyezik meg!
- Memória 32 – 47 című bájtjai: bit-címezhető memóriamező
 - 0 – 127 bitcím-tartomány
 - Logikai értékek optimális reprezentációja

Intel 8051

• Speciális regiszterek

- PSW
 - C, A, O, P: mint korábban
 - RS: regiszterkészlet választó
- IE
 - Megszakítások kollektív (EA) és egyedi (E., X.) engedélyezése és tiltása
- IP
 - Megszakítások prioritási sorrendje
 - Alacsony/magas: alacsony megszakítható magassal, fordítva nem
- TCON
 - Fő időzítők kezelése
- TMOD
 - Időzítők üzemmódja



← 8 bit →

Forrás: Tanenbaum

Nem kell megtanulni!

Intel 8051

- **További regiszterek**
 - 128-255 címtartományban
 - Akkumulátor: aritmetikai műveletekhez (240-es címen)
 - B/K portok
 - Áramellátással, soros port vezérléssel kapcsolatos regiszterek

AVR ISA

- **Jellemzői**

- Alacsony számítási igényű és energiafelhasználású beágyazott rendszerekben
- ATmega48, ATmega88, ATmega168, ...
- Egyféle üzemmód, nincs védett mód, egy program fut

- **Memóriamodell (ATmega168)**

- 16 KB program memória (Flash memória)
 - Rendszerbetöltő szekció
 - Csak innen futó kód módosíthatja a Flash memóriát (írhat be programot)
 - A gyártó digitális aláírással láthatja el a kódot
 - Alkalmazás szekció
- 1 KB adat memória (SRAM memória)
- Különálló címtartománnyal!

AVR ISA

- **Regiszterek**

- 32 darab 8 bites általános célú regiszter (R0 – R31)
 - Memóriába ágyazottak
 - 0. adatbájt R0 regiszterrel egyenértékű, ...
- Speciális célú regiszterek
 - Státusz regiszter (SREG): jelzőbitek
 - Megszakítás engedélyezés; fél-átvitel; előjel; túlszordulás; negatív; nulla; átvitel
 - Verem mutató (SP): 16 bites; 80 – 81. memóriacímen

- **Adat memória**

- 0 – 31 bájtok: R0 – R31 regiszterek
- 32 – 95 bájtok: 64 darab B/K eszköz regiszter
- 96 – 1023: adatterület

ISA-szint adattípusai

- **Hardver mit támogat**

- **Numerikus**

- Egész, lebegőpontos
 - Többféle kódolással (BCD, kettes komplement, IEEE 754, ...)

- **Nem numerikus**

- Logikai értékek
 - 0 (hamis) és 1 (igaz)
 - Jellemzően egész bájtól tárolva
 - Karakterláncok (sztringek)
 - ASCII, UTF, ...

- **Szoftveres megvalósítás**

- Ha nincs hardver támogatás
 - Komplex számok, sztringként leírt hosszú egész és tört számok, ...
 - Szabadon választható reprezentáció

ISA-szint adattípusai

A táblázatot nem kell megtanulni!

Típus	1 bit	8 bit	16 bit	32 bit	64 bit	128 bit
Bit	8					
Előjeles egész		P,C,U,A,8,M	P,C,U,A	P,C,U,A	C,U	
Előjel nélküli egész		P,C,U,A,M	P,C,U,A,M	P,C,U,A	C,U	
BCD		P,C				
Lebegőpontos				P,C,U,A	P,C,U,A	U

• Jelölés

- P (Pentium 4)
- C (Core i7)
- U (UltraSPARC III)
- 8 (8051)
- A (ARMv7)
- M (ATmega168)

• Előjeles egész

- Kettes komplementes ábrázolás

• Lebegőpontos

- IEEE 754 szabvány szerint

• BCD

- Csak az Intel processzorok esetén támogatott

ISA-szint utasításformátumai

- **Utasítás tartalma**

- Műveleti kód: mit kell csinálni
- Operandusok: min kell elvégezni
 - Memóriacím, regiszter(ek), kódba épített adat
- Prefix: utasítás hatásának esetleges módosítására
 - Sztringművelet ismétlésére, sín zárolására, ...

- **Utasítások hossza**

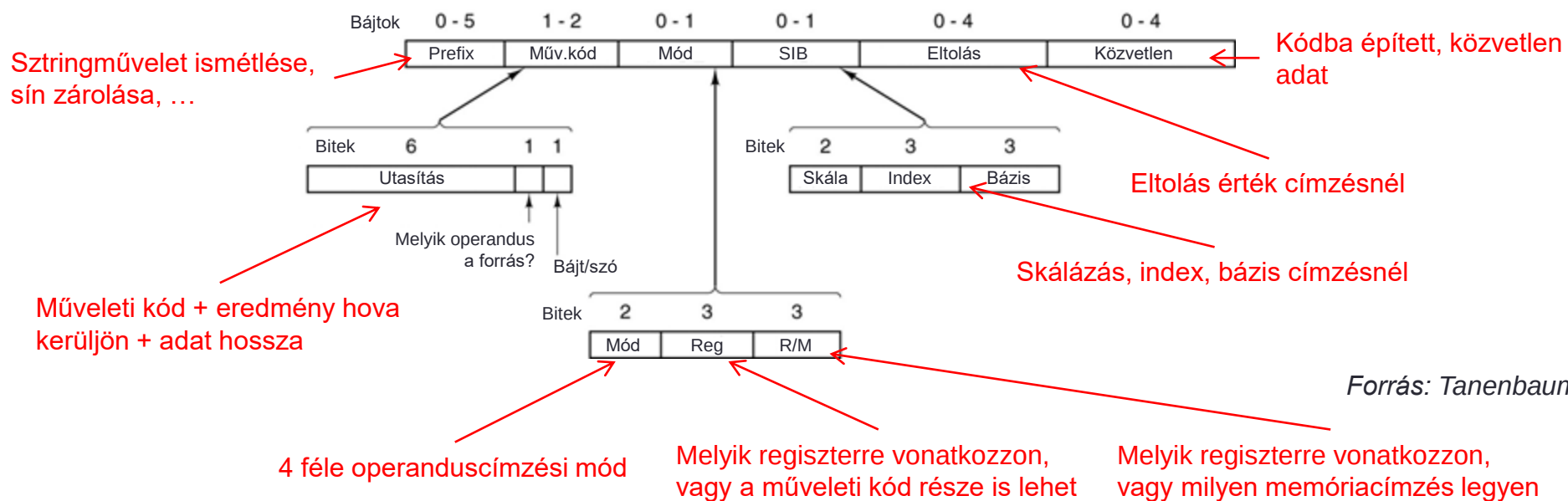
- Egyező: rögtön ismert a következő utasítás címe (kivéve ugrások esetén), de pazarlóbb a memóriával
- Változó: lassabb dekódolás
- Műveleti kód kiterjesztése
 - Változó számú bit írja le a kódot és az operandusokat
 - De egyértelműen dekódolhatónak kell lennie

P4 és Core i7 utasításformátumai

• Jellemzői

- Bonyolult, szabálytalan: 1-17 bájt közötti méret
- Hatféle, változó hosszúságú mező, több opcionális
- Régi ISA „foltozása” a kompatibilitás miatt
- Sok, komplex utasítás
- Nem lehet mindkét operandus memóriahivatkozás, legalább 1 regiszter kell

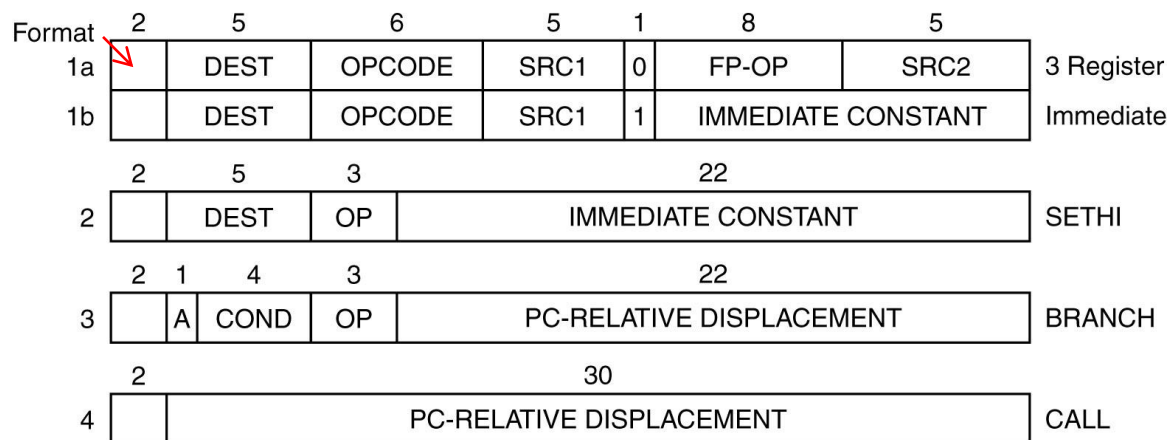
Az ábrát nem kell megtanulni!



UltraSPARC III utasításformátumai

• Jellemzői

- Minden utasítás 32 bites és szóhatárra igazított
- Aritmetikai művelet: két bemenő regiszter, egy eredmény; vagy egyik regiszter helyett 13 bites előjeles konstans
- Kevesebb, egyszerűbb művelet



Az ábrát nem kell megtanulni!

Három regiszter

-4096 és 4095 közötti érték

A SETHI utasításnak
22 bites operandus kell

Feltételes elágazás

Eljárás hívása: a 30 bites
érték négyszerese a cél

TI OMAP4430 utasításformátumai

- **Jellemzői**

- 16 és 32 bit hosszúságú, egyszerű utasítások
 - Pl. aritmetikai: két forrásregiszter, 1 célregiszter
 - 16 bites esetben „Thumb ISA”: csak két regiszter adható meg, bemenetként csak az első 8 regiszter adható meg
- Memóriában igazított helyen találhatók
- Predikált (feltételes) utasítások
 - Eredmény csak a feltétel teljesülése esetén tárolódik
- Ugrás operandusa csak 24 bites!
 - Csak néggyel osztható címre ugorhatunk → $\pm 2^{25}$ távolság érhető el
 - Nagyobb távolság esetén IP regisztert kell használni

TI OMAP4430 utasításformátumai

• 32 bites utasításformátumok

A táblázatot nem kell megtanulni!

31			2827			1615			87			0			Instruction type						
Cond	0	0	I	Opcode		S	Rn	Rd	Operand2						Data processing / PSR Transfer						
Cond	0	0	0	0	0	0	A	S	Rd	Rn	RS	1	0	0	1	Rm	Multiply				
Cond	0	0	0	0	1	U	A	S	RdHi	RdLo	RS	1	0	0	1	Rm	Long Multiply				
Cond	0	0	0	1	0	B	0	0	Rn	Rd	0	0	0	0	1	0	0	1	Rm	Swap	
Cond	0	1	I	P	U	B	W	L	Rn	Rd	Offset						Load/Store Byte/Word				
Cond	1	0	0	P	U	S	W	L	Rn	Register List								Load/Store Multiple			
Cond	0	0	0	P	U	1	W	L	Rn	Rd	Offset1	1	S	H	1	Offset2	Halfword transfer: Immediate offset				
Cond	0	0	0	P	U	0	W	L	Rn	Rd	0	0	0	0	1	S	H	1	Rm	Halfword transfer: Register offset	
Cond	1	0	1	L	Offset											Branch					
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	0	0	0	1	Rn	Branch Exchange
Cond	1	1	0	P	U	N	W	L	Rn	CRd	CPNum	Offset						Coprocessor data transfer			
Cond	1	1	1	0	Op1			CRn	CRd	CPNum	Op2	0	CRm			Coprocessor data operation					
Cond	1	1	1	0	Op1	L	CRn	Rd	CPNum	Op2	1	CRm			Coprocessor register transfer						
Cond	1	1	1	1	SWI Number											Software interrupt					

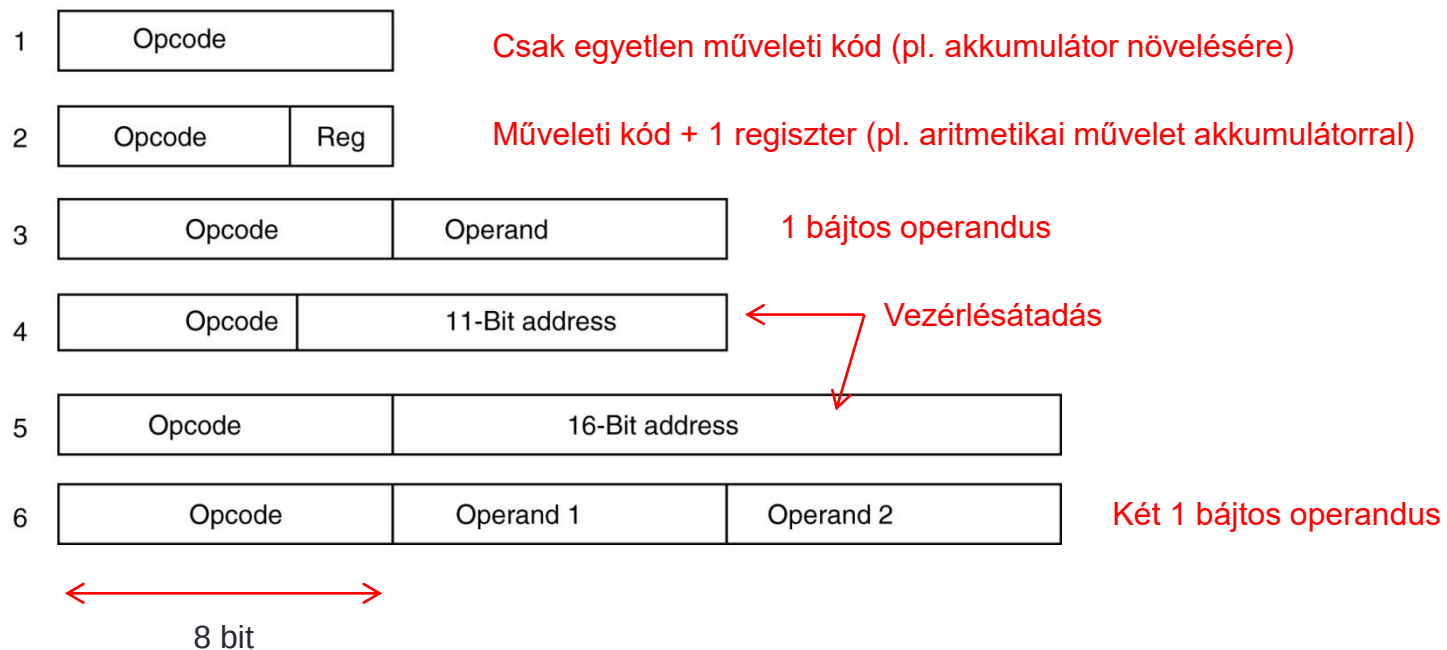
Intel 8051 utasításformátumai

• Jellemzői

- Hatféle egyszerű utasításforma

Format

Az ábrát nem kell megtanulni!



Forrás: Tanenbaum

ATmega168 AVR utasításformátumai

- **6 utasításformátum; 5 darab 16 bites és 1 darab 32 bites**
 - 1.: opkód + két regiszter
 - eredmény az egyik regiszterbe
 - 2.: opkód + regiszter
 - eredmény ugyanebbe a regiszterbe (negálás, inkrementálás)
 - 3.: opkód + 8 bites közvetlen (utasításba épített) adat + 1 regiszter
 - R16 – R31 közötti regiszter lehet
 - Csak 4 utasítás használhatja (2 bit marad opkód azonosításra)
 - 4.: opkód + 6 bites előjel nélküli közvetlen operandus
 - Betöltő/tároló utasítások; opkód határozza meg a használandó regisztert
 - 5.: opkód + 12 bites közvetlen eltolási érték
 - Ugró utasítás, PC relatív érték
 - 6.: opkód + 22 bites közvetlen eltolási érték
 - Ugró utasítás, PC relatív érték

Címzési módok

- **Feldolgozandó adat helyének megadása**
 - Operandus megadása közvetlenül az utasításban
 - Memóriabeli operandus teljes címének megadásával
 - Az operandus egy regiszter
 - ...
- Részletesebb tárgyalás az Assembly programozás kurzuson!

Vezérlési folyamat

- **Utasítások végrehajtási sorrendje**
 - **Szekvenciális vezérlés és elágazás**
 - Egymást követő utasítások végrehajtása
 - **Eljárások**
 - Programok strukturáltságához fontos
 - Eljárásprológus
 - Eljárás híváskor lejátszódó folyamat
 - Pl. veremkeret mentése, visszatérési cím a verembe, ...
 - Eljárásepilógus
 - Eljárásból való visszatéréskor lejátszódó folyamat
 - visszatérési cím beállítása, ...
 - **Rekurzív eljárás**
 - Önmagát hívja
 - Verem szükséges a megvalósításához

Vezérlési folyamat

- **Utasítások végrehajtási sorrendje**
 - **Csapdák**
 - Program által előidézett fontos, de csak ritkán bekövetkező események
 - Egész túlcsordulás, 0-val való osztás, lebegőpontos alul- vagy túlcsordulás
 - Bekövetkeztek a csapdakezelőre adódik a vezérlés
 - Nem kell minden utasítás után a hibalehetőségeket ellenőrizni
 - **Megszakítások**
 - B/K eszköz általi vezérlési folyamat módosítás
 - Megszakításkezelő
 - Bekövetkeztek ide adódik a vezérlés, végeztével a program fut tovább
 - Átlátszóság
 - A számítógép ugyanabba az állapotba kerül a megszakításkezelő után, mint előtte volt
 - Megszakítható megszakítások
 - Prioritási szintek bevezetése

RISC-V (kiejtése: RISC-5)

- **Probléma**

- ISA szint megvalósítása egy tervezett új processzorhoz nehéz és sok munkát igénylő folyamat
- A meglévők pénzért licenszelhetők (vagy egyáltalán nem)

- **Cél**

- RISC alapú ISA, amely nyílt szabványon alapul (University of California, Berkeley, USA)
- Szabadon felhasználható és skálázható különféle célokra

- **Hivatalos honlap**

- <https://riscv.org/>

- **Cikk (angol nyelven)**

- [*RISC-V is trying to launch an open-hardware revolution*](#)
- [*Qualcomm reveals it's making a RISC-V chip for Android wearables. It could be the first mass market RISC-V platform.*](#)

8051 fejlesztés

- **The 8051 Simulator for Teachers and Students**

- <https://www.edsim51.com/>
- 8051 mikrovezérlő programozásának tanulásához
- Periféria kommunikáció előzetes tesztelése tényleges hardver megépítése nélkül

- **MCU 8051 IDE**

- <https://sourceforge.net/projects/mcu8051ide/>
- Integrált fejlesztői környezet 8051-alapú processzorokhoz
- Assembly és C programozási nyelveken
- Szimulátor, assembler, kódszerkesztő és további eszközök

- **SDCC – Small Device C Compiler**

- <http://sdcc.sourceforge.net/>
- Standard C (ANSI C89, ISO C99, ISO C11) fordító mikrovezérlőkhöz (8051 és társai)

Pentium 4 utasításai

Adatmozgatás	
MOV DST, SRC	SRC másolása DST-be
PUSH SRC	SRC verembe helyezése
POP DST	Verem teteje DST-be
XCHG DS1, DS2	DS1 és DS2 felcserélése
LEA DST, SRC	SRC offszetcíme DST-be
CMOVcc DST, SRC	Feltételes adatmozgatás

Csak érdekesség,
nem kell megtanulni!

Aritmetikai	
ADD DST, SRC	SRC hozzáadása DST-hez
SUB DST, SRC	SRC kivonása DST-ből
MUL SRC	EAX előjel nélküli szorzása SRC-vel
IMUL SRC	EAX előjeles szorzása SRC-vel
DIV SRC	EDX:EAX előjel nélküli osztása SRC-vel
IDIV SRC	EDX:EAX előjeles osztása SRC-vel
ADC DST, SRC	SRC és carry hozzáadása DST-hez
SBB DST, SRC	SRC és carry kivonása DST-ből
INC DST	DST 1-el növelése
DEC DST	DST 1-el csökkentése
NEG DST	DST negálása

Pentium 4 utasításai

Binary coded decimal

DAA	Decimal adjust
DAS	Decimal adjust for subtraction
AAA	ASCII adjust for addition
AAS	ASCII adjust for subtraction
AAM	ASCII adjust for multiplication
AAD	ASCII adjust for division

Boolean

AND DST, SRC	Boolean AND SRC into DST
OR DST, SRC	Boolean OR SRC into DST
XOR DST, SRC	Boolean Exclusive OR SRC to DST
NOT DST	Replace DST with 1's complement

Shift/rotate

SAL/SAR DST, #	Shift DST left/right # bits
SHL/SHR DST, #	Logical shift DST left/right # bits
ROL/ROR DST, #	Rotate DST left/right # bits
RCL/RCR DST, #	Rotate DST through carry # bits

Test/compare

TEST SRC1, SRC2	Boolean AND operands, set flags
CMP SRC1, SRC2	Set flags based on SRC1 - SRC2

Transfer of control

JMP ADDR	Jump to ADDR
Jxx ADDR	Conditional jumps based on flags
CALL ADDR	Call procedure at ADDR
RET	Return from procedure
IRET	Return from interrupt
LOOPxx	Loop until condition met
INT n	Initiate a software interrupt
INTO	Interrupt if overflow bit is set

Strings

LODS	Load string
STOS	Store string
MOVS	Move string
CMPS	Compare two strings
SCAS	Scan Strings

Csak érdekesség, nem kell megtanulni!

Pentium 4 utasításai

Condition codes

STC	Set carry bit in EFLAGS register
CLC	Clear carry bit in EFLAGS register
CMC	Complement carry bit in EFLAGS
STD	Set direction bit in EFLAGS register
CLD	Clear direction bit in EFLAGS reg
STI	Set interrupt bit in EFLAGS register
CLI	Clear interrupt bit in EFLAGS reg
PUSHFD	Push EFLAGS register onto stack
POPF	Pop EFLAGS register from stack
LAHF	Load AH from EFLAGS register
SAHF	Store AH in EFLAGS register

Miscellaneous

SWAP DST	Change endianness of DST
CWQ	Extend EAX to EDX:EAX for division
CWDE	Extend 16-bit number in AX to EAX
ENTER SIZE,LV	Create stack frame with SIZE bytes
LEAVE	Undo stack frame built by ENTER
NOP	No operation
HLT	Halt
IN AL,PORT	Input a byte from PORT to AL
OUT PORT,AL	Output a byte from AL to PORT
WAIT	Wait for an interrupt

SRC = source
DST = destination

= shift/rotate count
LV = # locals

Csak érdekesség, nem kell megtanulni!

UltraSPARC III utasításai

Loads

LDSB ADDR,DST	Load signed byte (8 bits)
LDUB ADDR,DST	Load unsigned byte (8 bits)
LDSH ADDR,DST	Load signed halfword (16 bits)
LDUH ADDR,DST	Load unsigned halfword (16)
LDSW ADDR,DST	Load signed word (32 bits)
LDUW ADDR,DST	Load unsigned word (32 bits)
LDX ADDR,DST	Load extended (64-bits)

Stores

STB SRC,ADDR	Store byte (8 bits)
STH SRC,ADDR	Store halfword (16 bits)
STW SRC,ADDR	Store word (32 bits)
STX SRC,ADDR	Store extended (64 bits)

Arithmetic

ADD R1,S2,DST	Add
ADDCC “	Add and set icc
ADDC “	Add with carry
ADDCCC “	Add with carry and set icc
SUB R1,S2,DST	Subtract
SUBCC “	Subtract and set icc
SUBC “	Subtract with carry
SUBCCC “	Subtract with carry and set icc
MULX R1,S2,DST	Multiply
SDIVX R1,S2,DST	Signed divide
UDIVX R1,S2,DST	Unsigned divide
TADCC R1,S2,DST	Tagged add

Csak érdekesség, nem kell megtanulni!

UltraSPARC III utasításai

Shifts/rotates

SLL R1,S2,DST	Shift left logical (32 bits)
SLLX R1,S2,DST	Shift left logical extended (64)
SRL R1,S2,DST	Shift right logical (32 bits)
SRLX R1,S2,DST	Shift right logical extended (64)
SRA R1,S2,DST	Shift right arithmetic (32 bits)
SRAX R1,S2,DST	Shift right arithmetic ext. (64)

Miscellaneous

SETHI CON,DST	Set bits 10 to 31
MOVcc CC,S2,DST	Move on condition
MOVr R1,S2,DST	Move on register
NOP	No operation
POPC S1,DST	Population count
RDCCR V,DST	Read condition code register
WRCCR R1,S2,V	Write condition code register
RDPC V,DST	Read program counter

Boolean

AND R1,S2,DST	Boolean AND
ANDCC “	Boolean AND and set icc
ANDN “	Boolean NAND
ANDNCC “	Boolean NAND and set icc
OR R1,S2,DST	Boolean OR
ORCC “	Boolean OR and set icc
ORN “	Boolean NOR
ORNCC “	Boolean NOR and set icc
XOR R1,S2,DST	Boolean XOR
XORCC “	Boolean XOR and set icc
XNOR “	Boolean EXCLUSIVE NOR
XNORCC “	Boolean EXCL. NOR and set icc

Csak érdekesség, nem kell megtanulni!

UltraSPARC III utasításai

Transfer of control

BPcc ADDR	Branch with prediction
BPr SRC,ADDR	Branch on register
CALL ADDR	Call procedure
RETURN ADDR	Return from procedure
JMPL ADDR,DST	Jump and link
SAVE R1,S2,DST	Advance register windows
RESTORE “	Restore register windows
Tcc CC,TRAP#	Trap on condition
PREFETCH FCN	Prefetch data from memory
LDSTUB ADDR,R	Atomic load/store
MEMBAR MASK	Memory barrier

SRC = source register

DST = destination register

R1 = source register

S2 = source: register or immediate

ADDR = memory address

TRAP# = trap number

FCN = function code

MASK = operation type

CON = constant

V = register designator

CC = condition code set

R =destination register

cc = condition

r = LZ,LEZ,Z,NZ,GZ,GEZ

Csak érdekesség, nem kell megtanulni!

UltraSPARC III utasításai

Utasítás	Megoldás
MOV SRC , DST	OR SRC G0-al, tárolás DST-ben
CMP SRC1 , SRC2	SRC2 kivonása (SUBCC) SRC1-ből, tárolás G0-ban
TST SRC	ORCC SRC és G0 között, eredmény G0-ba
NOT DST	XNOR DST és G0 között
NEG DST	DST kivonása G0-ból, eredmény DST-be
INC DST	DST-hez 1 hozzáadása (ADD)
DEC DST	DST-ből 1 kivonása (SUB)
CLR DST	G0 vagyolása G0-al, eredmény DST-be
NOP	G0 felső bitjeinek nullázása (SETHI)
RET	JMPL %I7+8,%G0

Csak érdekesség, nem kell megtanulni!

8051 utasításai

Inst.	Description	ACC	Reg	Dir	@R	#	C	Bit
MOV	Move <i>src</i> to ACC		×	×	×	×		
MOV	Move <i>src</i> to register	×		×		×		
MOV	Move <i>src</i> to memory	×	×	×	×	×		
MOV	Move <i>src</i> to indirect RAM	×		×		×		
MOV	Move 16-bit constant to DPTR							
MOVC	Move code to ACC offset from DPTR							
MOVC	Move code to ACC offset from PC							
MOVX	Move external RAM byte to ACC				×			
MOVX	Move ext. RAM byte to ACC @DPTR							
MOVX	Move to ext. RAM byte from ACC				×			
MOVX	Move to ext. RAM byte from ACC @DPTR							
PUSH	Push <i>src</i> byte to stack			×				
POP	Pop stack byte to <i>dst</i>			×				
XCH	Exchange ACC and <i>dst</i>	×		×	×			
XCHD	Exchange low-order digit ACC and <i>dst</i>			×				
SWAP	Swap nibbles of <i>dst</i>	×						
ADD	Add <i>src</i> to ACC		×	×	×	×		

Csak érdekesség, nem kell megtanulni!

8051 utasításai

Inst.	Description	ACC	Reg	Dir	@R	#	C	Bit
ADDC	Add <i>src</i> to ACC with carry		×	×	×	×		
SUBB	Subtract <i>src</i> from ACC with borrow		×	×	×	×		
INC	Increment <i>dst</i>	×	×	×	×			
DEC	Decrement <i>dst</i>	×	×	×	×			
INC	DPTR							
MUL	Multiply							
DIV	Divide							
DA	Decimal adjust <i>dst</i>	×						
ANL	AND <i>src</i> to ACC		×	×	×	×		
ANL	AND ACC to <i>dst</i>			×				
ANL	AND immediate to <i>dst</i>			×				
ORL	OR <i>src</i> to ACC		×	×	×	×		
ORL	OR ACC to <i>dst</i>			×				
ORL	OR immediate to <i>dst</i>			×				

Csak érdekesség, nem kell megtanulni!

8051 utasításai

Inst.	Description	ACC	Reg	Dir	@R	#	C	Bit
XRL	XOR <i>src</i> to ACC		×	×	×	×		
XRL	XOR ACC to dst			×				
XRL	XOR immediate to dst			×				
CLR	Clear dst	×						
CPL	Complement dst	×						
RL	Rotate dst left	×						
RLC	Rotate dst left through carry	×						
RR	Rotate dst right	×						
RRC	Rotate dst right through carry	×						

Csak érdekesség, nem kell megtanulni!

8051 utasításai

Inst.	Description	ACC	Reg	Dir	@R	#	C	Bit
CLR	Clear bit						×	×
SETB	Set bit						×	×
CPL	Complement bit						×	×
ANL	AND <i>src</i> to carry							×
ANL	AND complement of <i>src</i> to carry							×
ORL	OR <i>src</i> to carry							×
ORL	OR complement of <i>src</i> to carry							×
MOV	Move <i>src</i> to carry							×
MOV	Move carry to <i>src</i>							×
JV	Jump relative if carry set							
JNC	Jump relative if carry not set							
JB	Jump relative if direct bit set							×
JNB	Jump relative if direct bit not set							×
JBC	Jump rel. if direct bit set and carry clear							×

Csak érdekesség, nem kell megtanulni!

8051 utasításai

Inst.	Description	ACC	Reg	Dir	@R	#	C	Bit
ACALL	Call subroutine (11-bit addr)							
LCALL	Call subroutine (16-bit addr)							
RET	Return from subroutine							
RETI	Return from interrupt							
SJMP	Short relative jump (8-bit addr)							
AJMP	Absolute jump (11-bit addr)							
LJMP	Absolute jump (16-bit addr)							
JMP	Jump indirect rel. to DPR+ACC							
JZ	Jump if ACC is zero							
JNZ	Jump if ACC is nonzero							
CJNE	Comp. <i>src</i> to ACC, jump unequal			×		×		
CJNE	Comp. <i>src</i> to immediate, jump unequal		×		×			
DJNZ	Decrement <i>dst</i> and jump nonzero							
NOP	No operation							

Csak érdekesség, nem kell megtanulni!

Assembly példaprogramok

- $N = I + J$ megvalósítása
- Pentium 4

Címke	Műv.kód	Operandusok ; Megjegyzések	
FORMULA:	MOV	EAX,I	; EAX regiszter = I
	ADD	EAX,J	; EAX regiszter = I + J
	MOV	N,EAX	; N = I + J
I	DD	3	; 4 bájtos terület, értéke 3
J	DD	4	; 4 bájtos terület, értéke 4
N	DD	0	; 4 bájtos terület, értéke 0

- Motorola 680x0

Címke	Műv.kód	Operandusok ; Megjegyzések	
FORMULA	MOVE.L	I,D0	; D0 regiszter = I
	ADD.L	J,D0	; D0 regiszter = I + J
	MOVE.L	D0,N	; N = I + J
I	DC.L	3	; 4 bájtos terület, értéke 3
J	DC.L	4	; 4 bájtos terület, értéke 4
N	DC.L	0	; 4 bájtos terület, értéke 0

Csak érdekesség, nem kell megtanulni!

Assembly példaprogramok

- $N = I + J$ megvalósítása
- UltraSPARC III

Címke	Műv.kód	Operandusok	; Megjegyzések
FORMULA:	SETHI	%HI(I),%R1	; R1 = I címének felső bitjei
	LD	[%R1+%LO(I)],%R1	; R1 = I
	SETHI	%HI(J),%R2	; R2 = J címének felső bitjei
	LD	[%R2+%LO(J)],%R2	; R2 = J
	NOP		; Várakozás J betöltésére
	ADD	%R1,%R2,%R2	; R2 = R1 + R2
	SETHI	%HI(N),%R1	; R1 = N címének felső bitjei
	ST	%R2,[%R1+%LO(N)]	; N = I + J tárolása
I:	.WORD 3		; 4 bájtos terület, értéke 3
J:	.WORD 4		; 4 bájtos terület, értéke 4
N:	.WORD 0		; 4 bájtos terület, értéke 0